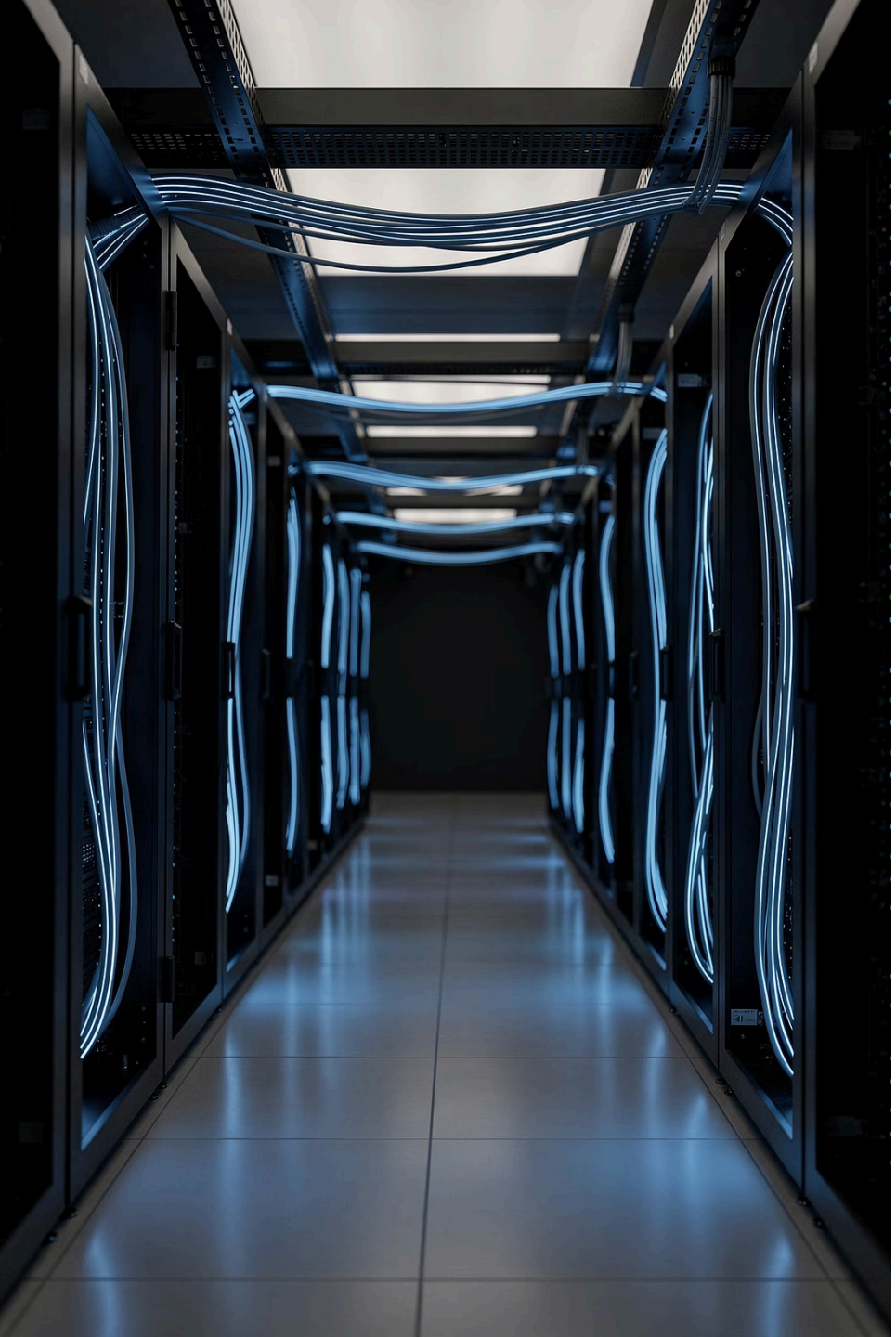


# Module 7: Dependency Injection & Services

Building Scalable and Maintainable Angular Applications

ANGULAR ARCHITECTURE | MODULE 7

# Module Overview

By the end of this module, you will be able to confidently apply Angular's dependency injection system in real-world, enterprise-scale applications.

01

## Understand DI

Grasp the core concept of Dependency Injection and why Angular uses it

02

## Manage Services

Learn how Angular creates, registers, and provides service instances

03

## Configure Providers

Control service scope at application, feature, and component levels

04

## Apply Best Practices

Use inject(), Injection Tokens, and service design patterns effectively

# What is Dependency Injection?

DI is a design pattern where Angular **provides** required dependencies to a class — instead of the class creating them itself.

### ❌ Without DI

- Components create their own dependencies
- Tight coupling between classes
- Difficult to test in isolation
- Hard to maintain at scale

### ✅ With DI

- Angular creates and manages dependencies
- Loose coupling between classes
- Services are reusable across components
- Easy to mock and unit test

Components should focus on UI. Angular manages object creation and dependency lifecycles.

# Why Dependency Injection?

Enterprise apps share common functionality across dozens of components. DI eliminates duplication and centralizes shared logic.

## Common Shared Services

**Auth**

**Employees**

**API**

**Notifications**

## Key Benefits

- **Code Reusability** — write once, inject anywhere
- **Loose Coupling** — components don't own their dependencies
- **Maintainability** — update logic in one place
- **Testability** — swap real services with mocks effortlessly
- **Centralized Logic** — business rules live in services, not components

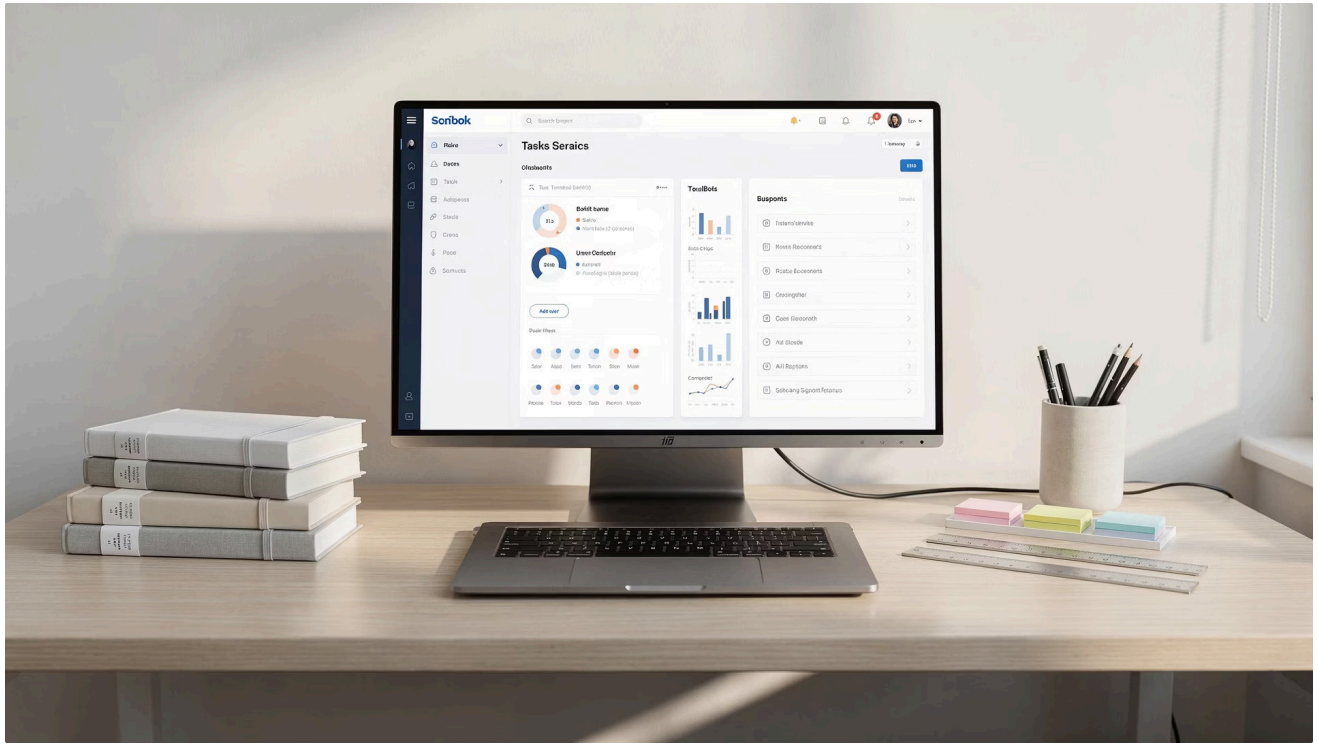


# Angular DI Architecture

When a component requests a dependency, Angular's injector resolves and provides the correct instance — automatically managing creation and lifecycle.




Components never manage object creation directly. The injector acts as a centralized registry — searching, instantiating, and delivering services on demand.




# What are Services?

A **Service** is a reusable class that encapsulates business logic, data access, or shared functionality — completely separate from the UI.

## Core Responsibilities

- Business logic and rules
- API communication
- Shared application state
- Utility and helper functions

Keep components focused on presentation. Move all business logic into services.

# Provider Configuration

Providers tell Angular **how and where** to create a service — controlling its lifetime, instance sharing, and memory footprint.

1

### Application Level

Single shared instance across the entire app. Use for Auth, Logging, Config.

2

### Feature Level

Scoped to a lazy-loaded module or route. Isolated from the rest of the app.

3

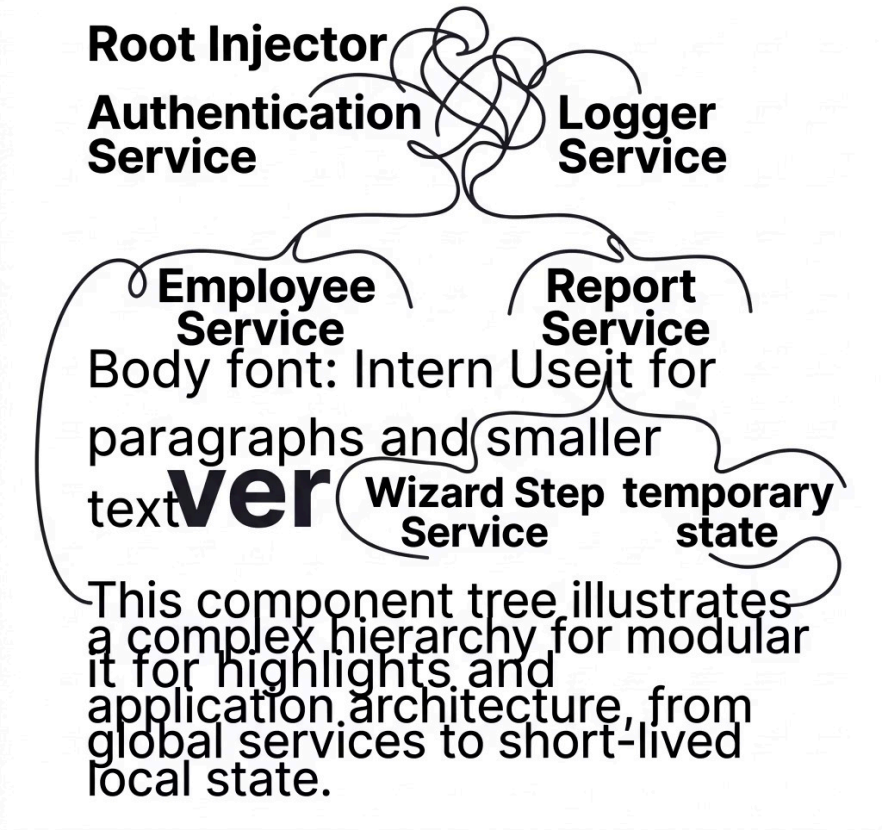
### Component Level

New instance per component. Use for wizard steps or temporary local state.

Default to application-level providers for shared services. Use narrower scopes only when isolation or independent instances are explicitly required.

# Injector Hierarchy

Angular's injectors are organized in a tree that mirrors your application structure — enabling scoped dependencies and optimized memory management.

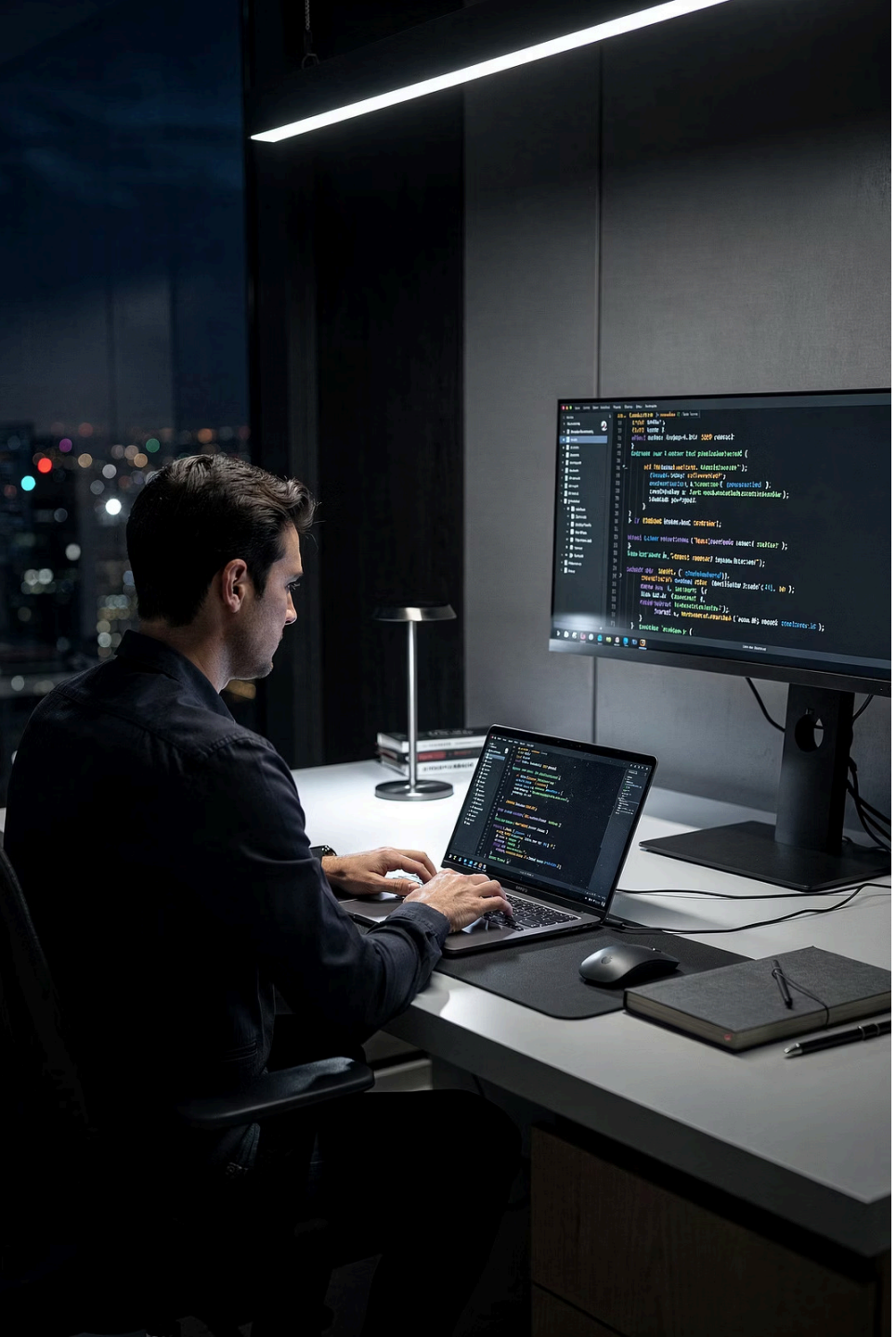



## Why It Matters

- **Scoped dependencies** — services only live where they're needed
- **Memory efficiency** — instances are created and destroyed with their scope
- **Feature isolation** — lazy modules get their own injector subtree

## Enterprise Example

- **Root** → AuthService (app-wide singleton)
- **Feature** → EmployeeService (HR module only)
- **Component** → WizardService (short-lived, per-form)



# The inject() API

Angular's `inject()` function is the modern, functional alternative to constructor injection — ideal for guards, interceptors, resolvers, and standalone APIs.

## Advantages

- Cleaner, more readable syntax
- First-class support for functional programming
- Required by many modern Angular APIs
- Works seamlessly in standalone components

## Best Use Cases

- Functional Route Guards
- HTTP Interceptors
- Utility and helper functions
- Standalone component dependencies

Use constructor injection for most components and services. Reach for inject() where it improves clarity or is required by Angular's API surface.

# Injection Tokens

When Angular can't identify a dependency by class type — such as a string, object, or primitive — **Injection Tokens** provide a unique, strongly-typed identifier.

## Common Use Cases

### API Base URL

Inject environment-specific endpoints without hardcoding

### Feature Flags

Toggle features per environment or tenant configuration

### App Settings

Centralize theme, locale, or external library config

## Key Benefits

- **Strong typing** — TypeScript enforces correct value types
- **Flexible configuration** — swap values per environment
- **No magic strings** — tokens act as explicit, referenceable keys
- **Testable** — override tokens easily in unit tests

```
const API_URL = new InjectionToken<string>('API_URL');
```

# Service Design Patterns

Well-designed services are predictable, reusable, and independently testable. Follow these patterns to keep your codebase maintainable at scale.



## Single Responsibility

One service owns one business capability. An AuthService handles auth — nothing else.



## Stateless by Default

Avoid storing mutable state unless necessary. Stateless services are easier to test and reuse.



## Centralize API Calls

All HTTP communication belongs in services — never in components. Components only consume data.



## No UI Logic

Services should never reference DOM, templates, or component state. Keep layers clean and separate.

# Best Practices

### 1 Inject, don't instantiate

Always rely on Angular's DI system — never use `new ServiceClass()` inside components.

### 2 One responsibility per service

Keep services focused. Split large services into smaller, purposeful units as your app grows.

### 3 Register at the right scope

Use root-level for singletons, feature-level for isolation, component-level for short-lived state.

### 4 Prefer constructor injection; use inject() intentionally

Constructor injection remains the standard. Use `inject()` only in functional APIs or when it genuinely improves clarity.

# Module Summary

## What We Covered

- **Dependency Injection**
- **Angular Services**
- **Provider Configuration**
- **Injector Hierarchy**
- **inject() API**
- **Injection Tokens**
- **Service Design Patterns & Best Practices**

## Up Next — Module 8

### Forms in Angular

- Template-Driven Forms
- Reactive Forms
- Validation strategies
- Dynamic Forms
- Signal-Based Forms